

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Математическое обеспечение и применение ЭВМ»

Курсовой проект

по дисциплине «Операционные системы»

на тему «Программа управления беспилотным летательным  
аппаратом»

ПГУ 09.03.04 – 06КП232.16 ПЗ

Направление подготовки – 09.03.04 Программная инженерия  
Профиль подготовки – Программная инженерия

Выполнил студент \_\_\_\_\_ Осин С.М.  
Группа \_\_\_\_\_ 23ВП2

Руководитель \_\_\_\_\_ Шашков Б.Д.  
к.т.н., профессор

Работа защищена с оценкой

Преподаватели

Дата защиты

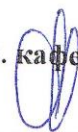
Министерство науки и высшего образования  
Российской Федерации

Пензенский государственный университет

Кафедра «Математическое обеспечение и применение ЭВМ»

«УТВЕРЖДАЮ»

Зав. кафедрой МО и ПЭВМ



Козлов А.Ю.

" " 2026 г.

## ЗАДАНИЕ

на курсовое проектирование по курсу

«Операционные системы»

Студенту Осину С.М. Группа 23ВІ2

Тема проекта: "Программа управления беспилотным летательным аппаратом"

Исходные данные (технические требования) на проектирование

Текущие координаты задаются в декартовой системе координат. Расстояние до БПЛА и высота в метрах. Программа полета представлена в виде цепочки команд изменения скорости, высоты и координат новой точки. Программа предварительно записана в файл. В пункте управления программа считывается и пересылается в БПЛА. Способ обмена информацией – сообщения.

Разработка должна проводиться на языке программирования C++.

Разработку выполнить с использованием операционной системы Linux.

Осуществить функциональное тестирование разработанной программы.

## Объём работы по курсу

### 1. Расчетная часть

---



---



---



---

### 2. Графическая часть

---



---



---



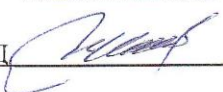
---

## Основные этапы выполнения курсового проекта

| N/N | Наименование этапа                                                  | Контрольные точки |
|-----|---------------------------------------------------------------------|-------------------|
| 1.  | Выдача задания                                                      |                   |
| 2.  | Разработка структуры программы и спецификаций на программные модули | 20 (5 неделя)     |
| 3.  | Разработка текстов программ                                         |                   |
| 4.  | Подготовка текстов программ на машинном носителе                    |                   |
| 5.  | Отладка программы                                                   | 40 (9 неделя)     |
| 6.  | Оформление пояснительной записки                                    |                   |
| 7.  | Защита курсовой работы                                              | 40 (15 неделя)    |
|     | <b>Всего</b>                                                        | <b>100</b>        |

Дата выдачи задания "3" февраля 2026 г.

Дата защиты проекта " " "

Руководитель Шашков Б.Д. 

Задание получил "3" февраля 2026 г.

Студент Осин С.М. 

## Реферат

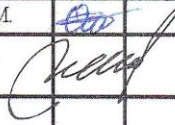
Пояснительная записка содержит 29 листа, 11 рисунков, 6 таблиц,  
6 использованных источников, 1 приложение

### ОПЕРАЦИОННЫЕ СИСТЕМЫ, LINUX, LINUX UBUNTU, ПРОЦЕССЫ, C++, СООБЩЕНИЯ, БЕСПИЛОТНЫЙ ЛЕТАТЕЛЬНЫЙ АППАРАТ.

Целью курсового проекта является разработка программы управления беспилотным летательным аппаратом.

Разработка проводилась на языке программирования C++ в редакторе Visual Studio Code на операционной системе Linux.

Осуществлено функциональное тестирование разработанного программного обеспечения, которое показало корректность его работы.

|           |      |             |                                                                                     |      |                                                                                   |                 |   |      |
|-----------|------|-------------|-------------------------------------------------------------------------------------|------|-----------------------------------------------------------------------------------|-----------------|---|------|
|           |      |             |                                                                                     |      | ПГУ 09.03.04 – 06КП232.16 ПЗ                                                      |                 |   |      |
| Изм.      | Лист | № докум.    | Подп.                                                                               | Дата |                                                                                   |                 |   |      |
| Разраб.   |      | Осин С.М.   |  |      | «Программа управления беспилотным летательным аппаратом»<br>Пояснительная записка | Лит.            |   | Лист |
| Пров.     |      | Пашков Б.Д. |  |      |                                                                                   |                 | 4 | 29   |
| Н. контр. |      |             |                                                                                     |      |                                                                                   | Группа<br>23ВП2 |   |      |
| Утв.      |      |             |                                                                                     |      |                                                                                   |                 |   |      |

## Содержание

|                                                        |    |
|--------------------------------------------------------|----|
| Введение.....                                          | 6  |
| 1. Постановка задачи и анализ предметной области ..... | 7  |
| 1.1. Основные понятия и определения.....               | 7  |
| 1.2. Постановка задачи .....                           | 7  |
| 1.3. Анализ требований .....                           | 8  |
| 1.3.1. Требования к интерфейсу пользователя .....      | 8  |
| 1.3.2. Требования к программным средства .....         | 8  |
| 2. Проектирование программы.....                       | 11 |
| 2.1. Модель интерфейса.....                            | 11 |
| 2.2. Структура программного обеспечения.....           | 11 |
| 3. Реализация программы.....                           | 13 |
| 3.1. Кодирование .....                                 | 13 |
| 3.2. Диаграмма компонентов.....                        | 13 |
| 4. Тестирование программы.....                         | 15 |
| 4.1. Функциональное тестирование.....                  | 15 |
| 4.2. Тестирование приложения.....                      | 15 |
| Заключение.....                                        | 21 |
| Список использованных источников .....                 | 22 |
| Приложение А.....                                      | 23 |

## Введение

Linux – это мощная многозадачная и многопользовательская операционная система, основанная на принципах открытого исходного кода. Она широко используется как на серверах, так и на персональных компьютерах, мобильных устройствах и встроенных системах. Одной из ключевых особенностей Linux является эффективное управление ресурсами и процессами, что позволяет системе одновременно выполнять множество задач. [1]

Одним из наиболее популярных дистрибутивов Linux является Ubuntu. Он основан на ядре Linux и ориентирован как на начинающих пользователей, так и на профессионалов. Ubuntu отличается удобным интерфейсом, регулярными обновлениями и широкой поддержкой сообщества. В системе реализованы все основные механизмы межпроцессного взаимодействия, характерные для Linux, а также предоставляются дополнительные инструменты и библиотеки для разработки и администрирования. [2]

В основе работы Linux лежит концепция процессов – отдельных выполняемых программ, которые изолированы друг от друга, но при этом могут взаимодействовать. Для организации такого взаимодействия используется механизм межпроцессного взаимодействия (IPC, Inter-Process Communication). Он позволяет процессам обмениваться данными и синхронизировать свои действия.

В Linux существует несколько способов организации обмена сообщениями между процессами. К ним относятся каналы (pipes), именованные каналы (FIFO), очереди сообщений, разделяемая память и сокеты. Каждый из этих механизмов имеет свои особенности и применяется в зависимости от требований к скорости, объёму передаваемых данных и уровню сложности взаимодействия. Например, каналы удобны для простого последовательного обмена данными между связанными процессами. Сообщения обеспечивают более высокий уровень абстракции при обмене данными. Они могут включать в себя не только сами данные, но и служебную информацию, такую как тип сообщения или его приоритет. [3]



## **1. Постановка задачи и анализ предметной области**

### **1.1. Основные понятия и определения**

Беспилотный летательный аппарат (БПЛА) – это летательный аппарат, на борту которого отсутствует экипаж. Управление им осуществляется дистанционно оператором или полностью в автоматическом режиме по заранее заложенному маршруту.

В операционной системе Linux взаимодействие между процессами реализуется с помощью механизмов межпроцессного взаимодействия, одним из которых являются сообщения.

Сообщение – это структурированный блок данных, который один процесс отправляет другому через специальный механизм операционной системы. [4]

В Linux для работы с очередями сообщений применяются системные вызовы, такие как создание очереди, отправка и получение сообщений, а также её удаление. [4]

Использование сообщений как способа взаимодействия между процессами удобно тем, что позволяет полностью разделить логику программ на независимые компоненты. Один процесс может выступать в роли отправителя данных, а другой – в роли обработчика, при этом они не обязаны знать внутреннюю реализацию друг друга.

### **1.2. Постановка задачи**

Основной задачей курсового проекта является разработка программы управления беспилотным летательным аппаратом. Координаты полёта беспилотника задаются в декартовой системе координат (x, y). Расстояние до БПЛА и высота в метрах. Программа полета должна быть представлена в виде цепочки команд: скорость полета, высота и координаты новой точки. Программа предварительно записывается в файл. Файл, в котором реализован пункт управления, считывается и пересылается в БПЛА. Способ обмена информацией – сообщения.

Программа должна быть разработана на языке программирования C++. Программное обеспечение должно быть отлажено, протестировано и

функционировать под управлением операционной системы Linux.

### 1.3. Анализ требований

#### 1.3.1. Требования к интерфейсу пользователя

Интерфейс программы будет представлять собой меню выбора типа беспилотника: мультироторный (пункт 1) и самолетный (пункт 2). При вводе любого другого символа – программа будет завершать работу. При завершении работы одного из типов БПЛА программа не будет завершать работу, а снова выводить вышеописанное меню до тех пор, пока пользователь не введёт любые другие символы.

#### 1.3.2. Требования к программным средствам

В результате поставленной задачи на курсовой проект, разработаем диаграмму вариантов использования, которая отображает основной функционал разрабатываемой программы. [5] Спроектированная диаграмма вариантов использования представлена на рисунке 1.

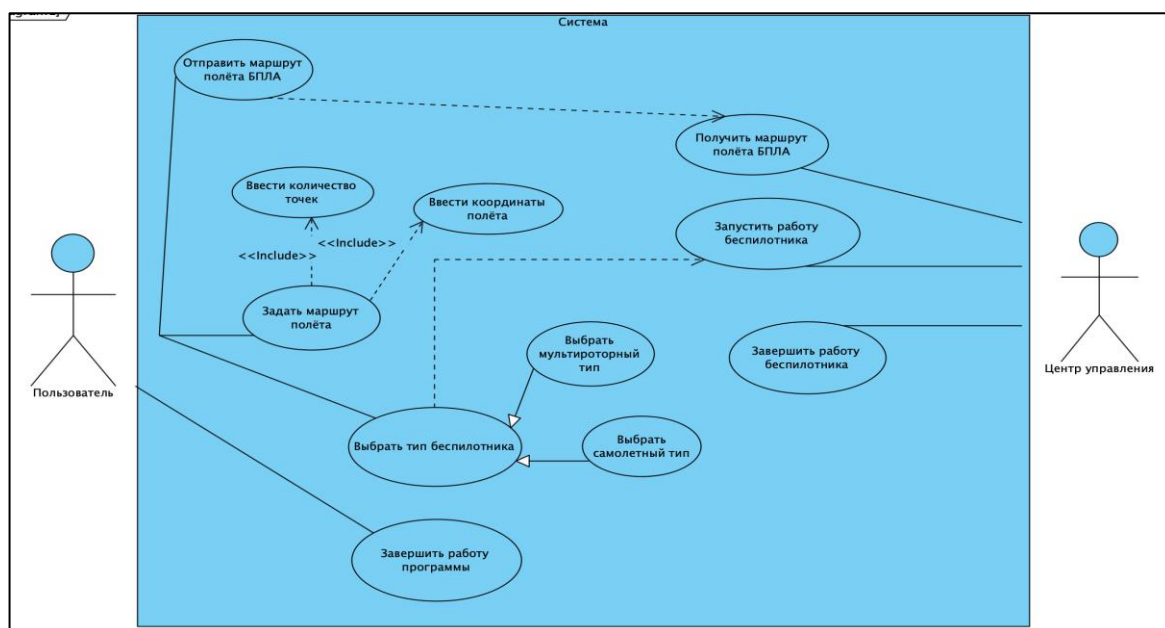


Рисунок 1 – Диаграмма вариантов использования

Описание вариантов использования представлены в таблицах 1-4.

Таблица 1 – Сценарий варианта использования «Выбрать тип беспилотника»

|                                                                 |
|-----------------------------------------------------------------|
| <b>Наименование:</b> выбрать тип беспилотника                   |
| <b>ID:</b> 1                                                    |
| <b>Краткое описание:</b> пользователь выбирает тип беспилотника |
| <b>Действующие лица:</b> пользователь                           |



Продолжение таблицы 1

|                                                                                                                                                                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Предусловие:</b> -                                                                                                                                                                                                                                                                               |
| <b>Основной поток:</b><br>1. Пользователь вводит символ для выбора типа беспилотника.<br>1.1. Если пользователь ввёл символ «1»<br>1.1.1. Выбран мультироторный тип БПЛА;<br>1.2. Иначе Пользователь ввёл символ «2»<br>1.2.1. Выбран самолетный тип БПЛА;<br>1.3. Иначе программа завершает работу |
| <b>Постусловие:</b> тип беспилотника выбран                                                                                                                                                                                                                                                         |

Таблица 2 – Сценарий варианта использования «Задать маршрут полёта»

|                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------|
| <b>Наименование:</b> задать маршрут полёта                                                                                         |
| <b>ID:</b> 2                                                                                                                       |
| <b>Краткое описание:</b> пользователь задаёт маршрут полёта БПЛА                                                                   |
| <b>Действующие лица:</b> пользователь                                                                                              |
| <b>Предусловие:</b> -                                                                                                              |
| <b>Основной поток:</b><br>1. Пользователь вводит количество точек (координат);<br>2. Пользователь вводит координаты полёта (x, y). |
| <b>Постусловие:</b> задан маршрут полёта БПЛА                                                                                      |

Таблица 3 – Сценарий варианта использования «получить маршрут полёта БПЛА»

|                                                                                                           |
|-----------------------------------------------------------------------------------------------------------|
| <b>Наименование:</b> получить маршрут полёта БПЛА                                                         |
| <b>ID:</b> 3                                                                                              |
| <b>Краткое описание:</b> после отправления координат центр управления получает эти координаты сообщениями |
| <b>Действующие лица:</b> центр управления                                                                 |
| <b>Предусловие:</b> сообщения получены                                                                    |

Продолжение таблицы 3

|                                                                                                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Основной поток:</b><br>1. Отправленные сообщениями от пользователя координаты появляются у центра управления;<br>2. Проверяется количество координат<br>2.1. Если количество равно нулю<br>2.1.1. Беспилотник не начинает работу<br>2.2. Иначе<br>2.2.1. Беспилотник начинает свой полёт. |
| <b>Постусловие:</b> маршруты полёта БПЛА получены                                                                                                                                                                                                                                            |

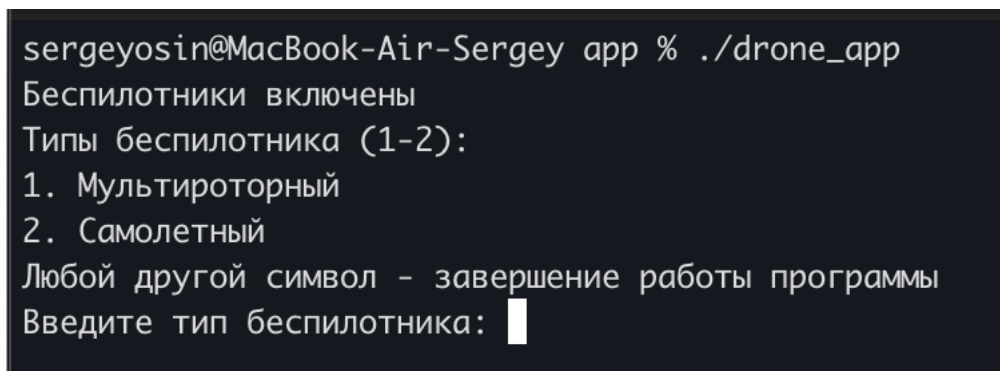
Таблица 4 – Сценарий варианта использования «запустить работу беспилотника»

|                                                                                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Наименование:</b> запустить работу беспилотника                                                                                                                                                                                         |
| <b>ID:</b> 4                                                                                                                                                                                                                               |
| <b>Краткое описание:</b> центр управления запускает работу беспилотника                                                                                                                                                                    |
| <b>Действующие лица:</b> центр управления                                                                                                                                                                                                  |
| <b>Предусловие:</b> количество координат больше нуля                                                                                                                                                                                       |
| <b>Основной поток:</b><br>1. После проверки количества первые координаты посылаются беспилотнику;<br>2. Беспилотник начинает полёт по координатам до тех пор, пока их количество не достигнет количества заданных координат пользователем. |
| <b>Постусловие:</b> беспилотник начал полёт по маршруту                                                                                                                                                                                    |

## 2. Проектирование программы

### 2.1. Модель интерфейса

Интерфейс представляет собой меню, в котором пользователю предлагается выбрать тип беспилотника: если пользователь ввёл 1, то запускается работа мультироторного БПЛА, при вводе 2 начинается работа самолетного БПЛА, а при вводе любого другого символа – программа завершает работу. Описанный интерфейс программы представлен на рисунке 2.



```
sergeyosin@MacBook-Air-Sergey app % ./drone_app
Беспилотники включены
Типы беспилотника (1-2):
1. Мультироторный
2. Самолетный
Любой другой символ - завершение работы программы
Введите тип беспилотника: █
```

Рисунок 2 – Интерфейс программы

### 2.2. Структура программного обеспечения

В ходе разработки программы реализовано 5 файлов: Main.cpp, который отвечает за запуск и взаимодействие с пользователем, Client.cpp, формирующий маршрут полёта БПЛА, и ControlCenter.cpp, выполняющий обработку и моделирование движения беспилотника.

В главной программе (Main.cpp) реализовано меню, которое работает, пока пользователь вводит 1 или 2. После выбора типа дрона создаётся очередь сообщений через msgget, затем процесс разделяется с помощью fork() на родительский и дочерний.

Родительский процесс формирует маршрут через createMessage, отправляет его в очередь (msgsnd), ожидает завершения дочернего процесса и удаляет очередь. Дочерний процесс получает сообщение через msgrcv, передаёт данные в processRoute, где выполняется моделирование полёта, после чего завершает работу.

В файлах клиента (Client.cpp и Client.h) реализована часть системы,

отвечающая за формирование сообщения с маршрутом беспилотника.

В заголовочном файле описаны структуры: Waypoint и Message.

```
struct Waypoint {  
    double x;  
    double y;  
};  
struct Message {  
    long mtype;  
    Waypoint points[100];  
    int count;  
    int droneType;  
};
```

Waypoint описывает координаты точки маршрута, а Message играет ключевую роль в работе всей программы: она является основным способом передачи данных между процессами – от клиента к центру управления.

В функции createMessage формируется готовое сообщение в зависимости от типа беспилотника. После заполнения всех данных структура возвращается и используется для передачи в центр управления через очередь сообщений.

В центре управления БПЛА (ControlCenter.cpp) реализована обработка маршрута БПЛА и моделирование его полёта в зависимости от типа дрона.

Сначала проверяется наличие маршрута в CheckCount. Затем в processRoute выбирается нужный вариант обработки: мультироторный или самолётный БПЛА.

Для мультироторного дрона моделируется движение по точкам с изменением высоты, зависанием и последующей посадкой. Для самолётного – прямолинейный полёт между точками с расчётом скорости и постепенным снижением до земли.

### 3. Реализация программы

#### 3.1. Кодирование

Разработаем диаграмму деятельности, демонстрирующую работу программы. [5] Разработанная диаграмма представлена на рисунке 3.

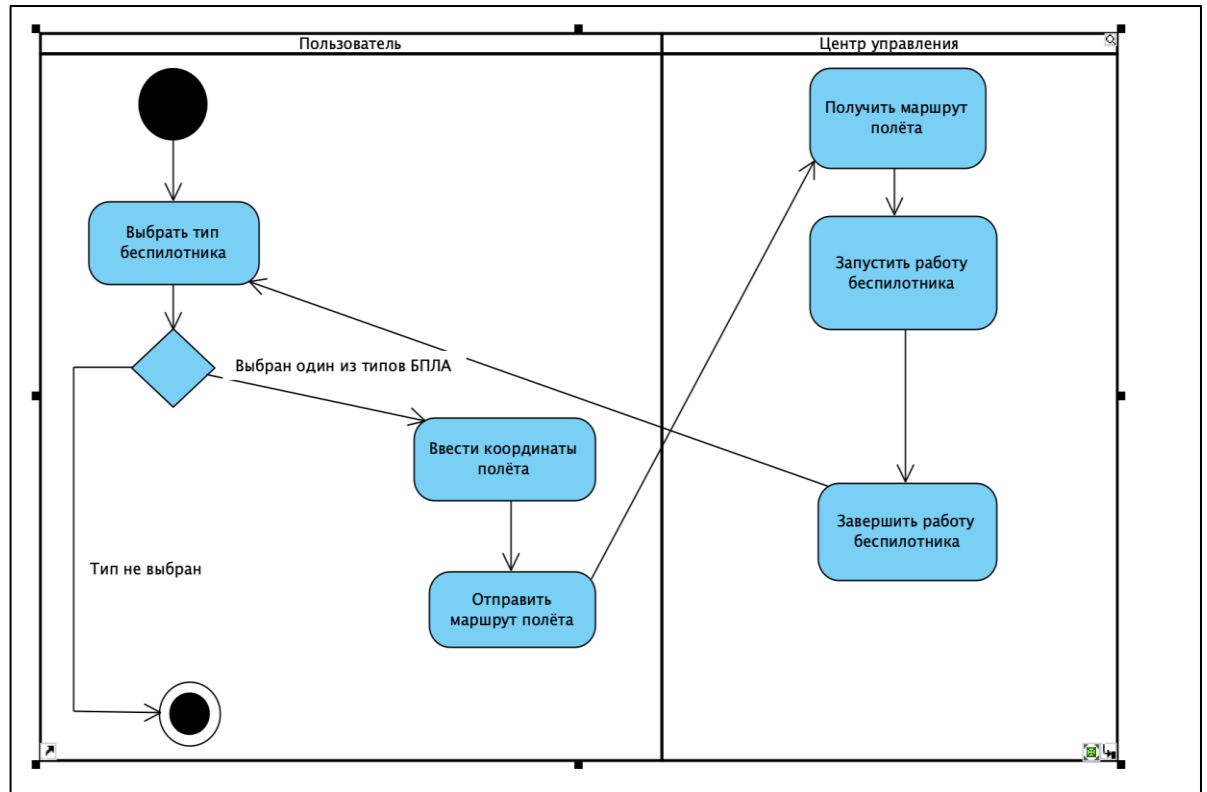


Рисунок 3 – Диаграмма деятельности

При запуске программы пользователь выбирает тип беспилотника: мультироторный или самолетный. Если тип не выбран (пользователь ввел не символы “1” или “2”), тогда программа завершает работу. Иначе вне зависимости от выбранного типа из файла считываются координаты полёта БПЛА (маршрут) и отправляются в центр управления. Центр управления получает сообщения о маршруте и запускает работу БПЛА. После прохождения маршрута БПЛА завершает работу и снова выводится меню.

#### 3.2. Диаграмма компонентов

Исходя из файлов, разработанных в результате проектирования программы, спроектируем диаграмму компонентов для демонстрации связи между компонентами проекта. Разработанная диаграмма представлена на рисунке 4.

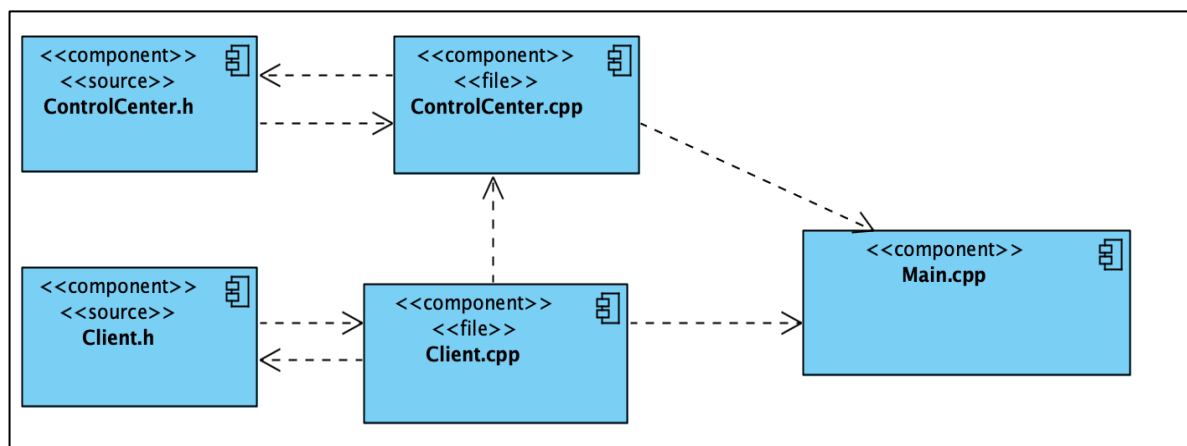


Рисунок 4 – Диаграмма компонентов

Описание компонентов представлено в таблице 5.

Таблица 5 – Описание компонентов программы

| Компонент         | Назначение                                                                             |
|-------------------|----------------------------------------------------------------------------------------|
| Main.cpp          | Главная программа, отображающая меню и запускающая выполнение приложения.              |
| ControlCenter.cpp | Центр управления БПЛА, обрабатывающий сообщения от пользователя                        |
| ControlCenter.h   | Заголовочный файл центра управления, содержащий объявления классов, функций и структур |
| Client.cpp        | Файл клиента, отправляющий координаты в центр управления                               |
| Client.h          | Заголовочный файл клиента, содержащий определения структур сообщений и точек           |



## 4. Тестирование программы

### 4.1. Функциональное тестирование

Функциональное тестирование – один из ключевых видов тестирования программного обеспечения, который фокусируется на проверке функциональности системы в соответствии с её требованиями и спецификациями. Данный вид тестирования направлен на то, чтобы убедиться, что приложение выполняет все предусмотренные функции, а также работает корректно с различными входными данными. [6]

Преимущества функционального тестирования:

- Проверка соответствия требованиям;
- Охват ключевой логики приложения;
- Близко к пользовательскому опыту;
- Подходит для разных уровней тестирования.

Недостатки функционального тестирования:

- Не проверяет нефункциональные аспекты;
- Трудоёмкость ручного тестирования;
- Ложные срабатывания;
- Ограниченная глубина проверки.

Исходя из функциональных требований в техническом задании, вышеперечисленных недостатков и преимуществ, в курсовом проекте будет проведено функциональное тестирование.

### 4.2. Тестирование приложения

Проведём функциональное тестирование разработанной программы, состоящее из 5 различных тестов.

Таблица 6 – Функциональное тестирование программы

| Состав теста                      | Ожидаемый результат        | Наблюдаемый результат                   |
|-----------------------------------|----------------------------|-----------------------------------------|
| В меню программы ввели символ 12. | Программа завершит работу. | Программа завершила работу. (рисунок 5) |

Продолжение таблицы 6

|                                                        |                                                                                        |                                                                          |
|--------------------------------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Зададим количество точек у первого типа БПЛА равным 0. | Выведет сообщение «Маршрут беспилотника не задан».                                     | Вывело соответствующее сообщение (рисунок 6).                            |
| Зададим количество точек у второго типа БПЛА равным 0. | Выведет сообщение «Маршрут беспилотника не задан».                                     | Вывело соответствующее сообщение (рисунок 7)                             |
| Зададим маршрут у первого типа БПЛА в виде 5 точек.    | БПЛА пролетит без ошибок все точки, выводя высоту и скорость полёта.                   | БПЛА пролетел маршрут с выводом высоты, скорости полёта (рисунок 8-9).   |
| Зададим маршрут у второго типа БПЛА в виде 6 точек.    | БПЛА пролетит все точки быстрее, чем первый тип, так как этот тип не зависит в воздухе | БПЛА пролетел все точки, выводя высоту и скорость полёта (рисунок 10-11) |

Результаты вышеописанных тестов представлены на рисунках 5-11.

```

Беспилотники включены
Типы беспилотника (1-2):
1. Мультироторный
2. Самолетный
Любой другой символ – завершение работы программы
Введите тип беспилотника: 12
Беспилотники отключены
Программа завершает работу

```

Рисунок 5 – Результат ввода символа «12»

```
Беспилотники включены
Типы беспилотника (1-2):
1. Мультироторный
2. Самолетный
Любой другой символ – завершение работы программы
Введите тип беспилотника: 1
Тип: Мультироторный БПЛА
Маршрут полёта беспилотника не задан
Центр управления завершил работу
```

Рисунок 6 – Работа БПЛА первого типа при 0 заданных точек

```
Беспилотники включены
Типы беспилотника (1-2):
1. Мультироторный
2. Самолетный
Любой другой символ – завершение работы программы
Введите тип беспилотника: 2
Тип: Самолетный БПЛА
Маршрут полёта беспилотника не задан
Центр управления завершил работу
```

Рисунок 7 – Работа БПЛА второго типа при 0 заданных точек

Введите тип беспилотника: 1  
Тип: Мультироторный БПЛА  
Маршрут беспилотника задан 5 точками  
Дрон начал маршрут

Дрон летит на точку (0,8)  
Изменение высоты до 8...  
Скорость по вертикали: 8 м/с

Дрон летит на точку (6,9)  
Изменение высоты до 9...  
Скорость по вертикали: 1 м/с  
Полёт по горизонтали к  $X = 6$   
Скорость по горизонтали: 6 м/с  
Дрон застыл в воздухе

Дрон летит на точку (3,11)  
Изменение высоты до 11...  
Скорость по вертикали: 2 м/с  
Полёт по горизонтали к  $X = 3$   
Скорость по горизонтали: 3 м/с  
Дрон застыл в воздухе

Рисунок 8 – Начало полёта БПЛА первого типа

Дрон летит на точку (8,12)  
Изменение высоты до 12...  
Скорость по вертикали: 1 м/с  
Полёт по горизонтали к  $X = 8$   
Скорость по горизонтали: 5 м/с  
Дрон застыл в воздухе  
Начало посадки  
Снижение с высоты 12 до 0  
Скорость снижения: 12 м/с  
Дрон приземлился  
Маршрут завершён.  
Центр управления завершил работу

Рисунок 9 – Конец полёта БПЛА первого типа

Введите тип беспилотника: 2  
Тип: Самолетный БПЛА  
Маршрут беспилотника задан 6 точками  
Дрон начал маршрут

Дрон летит на точку (1 4)  
Прямой полёт к точке (1, 4)  
Скорость: 6.18466 м/с

Дрон летит на точку (3 9)  
Прямой полёт к точке (3, 9)  
Скорость: 8.07775 м/с

Дрон летит на точку (4 12)  
Прямой полёт к точке (4, 12)  
Скорость: 4.74342 м/с

Дрон летит на точку (6 14)  
Прямой полёт к точке (6, 14)  
Скорость: 4.24264 м/с

Рисунок 10 – Начало полёта БПЛА второго типа

```
Дрон летит на точку (8 16)
Прямой полёт к точке (8, 16)
Скорость: 4.24264 м/с

Начало снижения
Высота: 14 м
Высота: 12 м
Высота: 10 м
Высота: 8 м
Высота: 6 м
Высота: 4 м
Высота: 2 м
Высота: 0 м
Касание земли
Пробег по полосе
Беспилотник приземлился
Маршрут завершён.
Центр управления завершил работу
```

Рисунок 11 – Конец полёта БПЛА второго типа

В ходе выполнения функционального тестирования несовпадения ожидаемого и наблюдаемого результата не выявлены. Следовательно, можно сделать вывод, что приложение работает корректно.

## **Заключение**

В результате выполнения курсового проекта была разработана программа управления беспилотным летательным аппаратом на языке программирования C++ на операционной системе Linux.

В процессе разработки были последовательно выполнены все ключевые этапы разработки ПО – от постановки задачи и анализа предметной области до реализации и тестирования. Это позволило создать программу, соответствующую техническим требованиям. Проведённое функциональное тестирование подтвердило корректность работы основных функций и стабильность программы в рамках поставленных задач.

Для моделирования программных средств был использован язык UML. Графическая часть проекта включает: диаграмма вариантов использования, диаграмма компонентов и диаграмма деятельности. Разработка осуществлена на языке C++ в операционной системе Linux Ubuntu.

### **Список использованных источников**

1. Стивенс У.Р. UNIX: взаимодействие процессов. – СПб.: Питер, 2002. – 576 с.
2. Робачевский А.М. Операционная система UNIX. - СПб.: BHV - Санкт-Петербург, 1997.
3. UNIX. Профессиональное программирование, 3-е издание. – Пер. с англ. – СПб.: Символ-Плюс, 2014. – 1104 с.
4. Фуско Дж. Linux. Руководство программиста. – СПб.: Питер, 2011.
5. Файлуер, М. UML. Основы, 3-е издание. – Пер. с англ./М. Фаулер – Санкт-Петербург: Символ – Плюс, 2004. – 192 с.
6. Пероцкая, В. Н. Основы тестирования программного обеспечения учеб. пособие / В. Н. Пероцкая, Д. А. Градусов; Владим. гос. ун-т им. А. Г. и Н. Г. Столетовых. – Владимир: Изд-во ВлГУ, 2017–100 с.



**Приложение А.  
КОД ПРОГРАММЫ  
(обязательное)**

## Main.cpp

```
#include <iostream>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/types.h>
#include "Client.h"
#include "ControlCenter.h"

using namespace std;

void displayForm() {
    string symb = "1";
    cout<<"Беспилотники включены\n";
    while (symb == "1" || symb == "2") {
        cout << "Типы беспилотника (1-2):\n";
        cout << "1. Мультироторный\n";
        cout << "2. Самолетный\n";
        cout << "Любой другой символ - завершение работы программы\n";
        cout << "Введите тип беспилотника: ";
        cin >> symb;
        if (symb == "1" || symb == "2") {

            int type = symb[0] - '0';

            int msqid = msgget(IPC_PRIVATE, 0666 | IPC_CREAT);
            if (msqid == -1) {
                perror("Ошибка получения сообщения");
                return;
            }

            pid_t pid = fork();

            if (pid < 0) {
                cerr << "Ошибка fork\n";
                msgctl(msqid, IPC_RMID, NULL);
                return;
            }

            if (pid > 0) {
                Message msg=createMessage(type);
                msg.mtype=1;
                if (msgsnd(msqid, &msg, sizeof(Message) -
sizeof(long), 0) == -1) {
                    perror("Ошибка отправки сообщения");
                }

                wait(NULL);

                msgctl(msqid, IPC_RMID, NULL);
            }
        }
    }
}
```

```

        cout << "Центр управления завершил работу\n\n";
    }
    else {
        Message msg;

        if (msgrcv(msqid, &msg, sizeof(Message) -
sizeof(long), 1, 0) == -1) {
            perror("Ошибка получения сообщения");
            exit(1);
        }
        processRoute(msg);
        exit(0);
    }
}

}
cout<<"Беспилотники отключены\n";
cout << "Программа завершает работу\n";
}

int main() {
    displayForm();
    return 0;
}

```

### Client.h

```

#ifndef CLIENT_H
#define CLIENT_H

struct Waypoint {
    double x;
    double y;
};

struct Message {
    long mtype;
    Waypoint points[100];
    int count;
    int droneType;
};

Message createMessage(int dronetype);

#endif

```

### Client.cpp

```

#include "Client.h"

Message msg;

Message createMessage(int droneType) {

```

```

msg.mtype=1;
msg.droneType = droneType;
if (droneType==1){
    msg.count = 5;
    msg.points[0] = {0, 0};
    msg.points[1] = {0, 8};
    msg.points[2] = {6, 9};
    msg.points[3] = {3, 11};
    msg.points[4] = {8,12};
}
else{
    msg.count = 6;
    msg.points[0] = {0, 0};
    msg.points[1] = {1,4};
    msg.points[2] = {3,9};
    msg.points[3] = {4,12};
    msg.points[4]={6,14};
    msg.points[5]={8,16};
}
return msg;
}

```

### **ControlCenter.h**

```

#ifndef CONTROL_CENTER_H
#define CONTROL_CENTER_H

#include "Client.h"

void processRoute(const Message& msg);

bool CheckCount(const Message&msg);

void processMultirotor(const Message& msg);

void processFixedWing(const Message& msg);
#endif

```

## ControlCenter.cpp

```
#include <iostream>
#include <unistd.h>
#include <cmath>
#include "ControlCenter.h"

using namespace std;

bool CheckCount(const Message&msg){
    if (msg.count==0){
        cout<<"Маршрут полёта беспилотника не задан\n";
        return false;
    }
    return true;
}

void processMultirotor(const Message& msg) {
    cout << "Тип: Мультироторный БПЛА\n";
    if (!CheckCount(msg)) return;
    cout<<"Маршрут беспилотника задан " << msg.count<< "
точками\n";
    cout << "Дрон начал маршрут\n";
    for (int i = 0; i < msg.count - 1; i++) {
        Waypoint a = msg.points[i];
        Waypoint b = msg.points[i + 1];

        cout<<"\nДрон летит на точку ("<< b.x<<","<<b.y<<")\n";
        sleep(1);

        double dy = b.y - a.y;

        if (dy != 0) {
            cout << "Изменение высоты до " << b.y << "... \n";
            cout << "Скорость по вертикали: " << abs(dy) << "
м/с\n";
            sleep(1);
        }

        double dx = b.x - a.x;

        if (dx != 0) {
            cout << "Полёт по горизонтали к X = " << b.x << "\n";
            cout << "Скорость по горизонтали: " << abs(dx) << "
м/с\n";
            cout<<"Дрон застыл в воздухе\n";
            sleep(3);
        }
    }

    Waypoint last = msg.points[msg.count - 1];

    if (last.y > 0) {
        cout << "Начало посадки\n";
    }
}
```

```

        cout << "Снижение с высоты " << last.y << " до 0\n";
        cout << "Скорость снижения: " << last.y << " м/с\n";
        sleep(2);
    }
    cout<<"Дрон приземлился\n";

    cout << "Маршрут завершён.\n";
}

void processFixedWing(const Message& msg) {
    cout << "Тип: Самолетный БПЛА\n";
    if (!CheckCount(msg)) return;
    cout<<"Маршрут беспилотника задан " << msg.count<< "
точками\n";
    cout << "Дрон начал маршрут\n";
    for (int i = 0; i < msg.count - 1; i++) {
        Waypoint a = msg.points[i];
        Waypoint b = msg.points[i + 1];

        cout<<"\nДрон летит на точку (" << b.x<<" " <<b.y<<")\n";
        sleep(1);

        double dx = b.x - a.x;
        double dy = b.y - a.y;

        double distance = sqrt(dx * dx + dy * dy);

        double speed = distance * 1.5;

        cout << "Прямой полёт к точке (" << b.x << ", " << b.y <<
")\n";
        cout << "Скорость: " << speed << " м/с\n\n";

        sleep(1);
    }
    Waypoint last = msg.points[msg.count - 1];

    if (last.y > 0) {
        cout << "Начало снижения\n";

        double currentY = last.y;

        while (currentY > 0) {
            double step = min(2.0, currentY);
            currentY -= step;

            cout << "Высота: " << currentY << " м\n";
            sleep(1);
        }

        cout << "Касание земли\n";
        cout << "Пробег по полосе\n";
        sleep(1);
    }
}

```

```
    }  
    cout<<"Беспилотник приземлился\n";  
  
    cout << "Маршрут завершён.\n";  
}  
  
void processRoute(const Message& msg) {  
  
    (msg.droneType==1)?processMultirotor(msg):processFixedWing(msg);  
}
```